

Fase 3 – Rediseño de arquitectura

María Alexandra Jiménez Suárez

Juan José Álvarez Restrepo

Carolina Ramírez Lotero

Natalia Giraldo Morales

Universidad Pontificia Bolivariana

Facultad de Ingeniería

Cesar Augusto López Gallego

10/08/2026

1. Propósito del documento

Este documento consolida los análisis y las decisiones arquitectónicas efectuadas para el rediseño del sistema de gestión de la farmacia. Se pretende una arquitectura TO-BE que mantenga el comportamiento observable del sistema original, y al mismo tiempo sea más sostenible para responder a las solicitudes de cambio planteadas.

Las decisiones se basan en los hallazgos identificados durante el diagnóstico del código y el análisis de las tres solicitudes de cambio. La arquitectura propuesta debe guardar coherencia entre el diseño UML y la implementación posterior.

2. Solicitudes de cambio consideradas

Código	Solicitud de cambio	Área principal afectada
SC-1	Incorporar cosméticos y productos comestibles, además de productos farmacéuticos.	Jerarquía y creación de productos
SC-2	Incorporar servicios de farmacia como inyectología, cambio de vendajes y curaciones básicas.	Modelo de dominio, operaciones comerciales y movimientos
SC-3	Incorporar convenios con diferentes entidades para descuentos y crédito.	Clientes, descuentos, convenios y operaciones comerciales

3. Tabla de decisiones arquitectónicas

Las decisiones de esta sección se construyen a partir de los hallazgos identificados en la Fase 1 y del impacto de las tres solicitudes de cambio analizadas en la Fase 2. En esta etapa se consideran las tres solicitudes para determinar qué aspectos de la arquitectura deben ser capaces de evolucionar. La sección TO-BE posterior establecerá cuáles de estas decisiones serán materializadas finalmente en el diseño y código.

ID	Principio	Elemento actual	Hallazgo / situación	Decisión arquitectónica
D-SC1-01	SRP	ServicioProducto	La clase concentra gestión de productos, carga desde archivos, verificación de stock, verificación de vencimiento y coordinación de eventos. H-01.	Separar las responsabilidades de gestión del catálogo, persistencia/carga de productos y evaluación de reglas de inventario/notificación, de manera que cada componente tenga una razón de cambio claramente definida.
D-SC1-02	OCP	ServicioProducto.CarregarDesdeArchivo()	La carga crea directamente MedicamentoCapsula, por lo que incorporar cosméticos,	Desacoplar la lectura de los datos de la creación de tipos concretos de

			comestibles u otro tipo de producto exige modificar la lógica existente. H-05. En SC-1 la carga y creación son los principales puntos de impacto.	productos mediante un mecanismo extensible de creación. La incorporación de nuevos tipos deberá realizarse mediante extensión y no modificando la lógica central de carga.
D-SC1-03	OCP + LSP	Jerarquía Producto → Medicamento → MedicamentoCapsula / MedicamentoLiquido	SC-1 requiere incorporar cosméticos y productos comestibles. Estos conceptos no son medicamentos. La jerarquía farmacéutica existente no debe forzarse para representar nuevos productos.	Mantener la jerarquía de medicamentos existente y permitir que las nuevas familias de productos sean especializaciones directas de Producto cuando corresponda. Las nuevas subclases deberán respetar el contrato de Producto.
D-SC1-04	OCP + SRP	Producto.MostrarInformacion() y presentación en Program.cs	Producto posee un punto de extensión mediante MostrarInformacion(), pero actualmente las subclases no lo aprovechan. La presentación de los productos se resuelve además mediante lógica específica en Program.cs. H-07.	Aprovechar el punto de extensión existente para que la información específica de cada tipo de producto pueda resolverse polimórficamente, evitando que el programa principal tenga que incorporar lógica específica para cada nuevo tipo de producto.
D-SC2-01	LSP	Producto / nuevo concepto Servicio	SC-2 incorpora servicios que pueden tener precio, pero no poseen naturalmente stock físico, stock mínimo ni fecha de vencimiento. Heredar Servicio de Producto obligaría a representar propiedades que no corresponden al dominio.	Separar conceptualmente productos físicos y servicios. Servicio no deberá heredar de Producto únicamente para reutilizar atributos o facilitar la venta. Se deberá definir una abstracción común únicamente para los comportamientos que realmente compartan.

D-SC2-02	SRP LSP	+	Movimiento y operaciones comerciales	Movimiento contiene directamente un Producto. La venta actual disminuye stock y registra un movimiento, por lo que una prestación de servicio no puede representarse directamente mediante la estructura actual.	Separar la representación de una operación comercial de las características específicas de un producto físico, permitiendo registrar operaciones sobre productos y servicios sin introducir atributos o comportamientos artificiales en Servicio.
D-SC2-03	SRP OCP	+	Reglas de stock y vencimiento en ServicioProducto	Las verificaciones actuales recorren Producto y aplican reglas de stock y vencimiento. Estas reglas no son aplicables a una prestación de servicio.	Separar las reglas aplicables exclusivamente a productos físicos de la gestión general de los elementos comercializables, evitando que la incorporación de servicios obligue a introducir excepciones dentro de las reglas existentes.
D-SC3-01	SRP OCP	+	Cliente y reglas de beneficios	Cliente actualmente contiene los puntos y AcumularPuntos() aplica directamente una regla fija. SC-3 introduce condiciones comerciales asociadas a diferentes convenios.	Mantener la identidad básica de Cliente independiente de las reglas comerciales y separar las reglas de beneficios, descuentos y condiciones asociadas a convenios para permitir su evolución sin modificar la entidad principal.
D-SC3-02	OCP LSP	+	Cliente y nuevo concepto Convenio	Un convenio representa una condición comercial asociada a un cliente y no un nuevo tipo de cliente. Crear subclases de Cliente para empresas, bancos, cooperativas, universidades, etc., produciría una jerarquía basada en una característica que puede variar independientemente.	Modelar Convenio como un concepto independiente relacionado con Cliente, utilizando composición/asociación en lugar de crear una subclase de Cliente por cada tipo de convenio.

D-SC3-03	OCP SRP	+	IDescuento ServicioDescuento	/	Existe una abstracción IDescuento, pero ServicioDescuento solo implementa un descuento fijo del 10 % y actualmente no está integrada al flujo de negocio. SC-3 requiere diferentes condiciones según el convenio.	Utilizar una abstracción para representar las reglas de descuento y permitir incorporar diferentes políticas de descuento asociadas a los convenios sin modificar el componente que coordina la operación comercial.
D-SC3-04	SRP OCP	+	Movimiento operación de venta	/	El movimiento actual no representa descuento, crédito ni información del convenio. SC-3 requiere que las operaciones realizadas bajo un convenio puedan conservar la información comercial correspondiente.	Extender la representación de la operación comercial para separar los datos propios del movimiento de las reglas de descuento y crédito, permitiendo incorporar información de convenios sin concentrar todas las reglas en Movimiento.
D-GEN-01	SRP		ServicioCliente		La clase combina gestión de clientes, carga desde archivos, acumulación de puntos y emisión de eventos. H-02.	Separar la gestión del cliente de la persistencia, las reglas de puntos y la notificación, manteniendo en cada componente una responsabilidad relacionada con una única razón de cambio.
D-GEN-02	SRP		Producto		La entidad de dominio contiene MostrarInformacion(), que realiza salida directa mediante consola. H-03.	Separar la representación y comportamiento del dominio de la responsabilidad de presentación, evitando que la entidad dependa de un mecanismo concreto de salida.
D-GEN-03	SRP		ServicioUsuario		La clase administra usuarios, carga datos desde archivos y coordina la autenticación. H-04. El equipo aceptó el hallazgo con la precisión de que la autenticación se encuentra delegada en AspectoAutenticacion.	Separar la carga/persistencia de usuarios (IRepositorioUsuario / RepositorioUsuarioArchivo) de la coordinación de autenticación. La autenticación se extrae a ServicioAutenticacion,

				clase independiente que recibe la lista de usuarios y coordina el proceso delegando en AspectoAutenticacion. ServicioUsuario queda con una única responsabilidad: custodiar la colección de usuarios.
D-GEN-04	LSP	Persona → Cliente / Usuario	La Fase 1 no encontró una violación de LSP. Las subclases conservan el contrato de la superclase y agregan información propia.	Conservar la jerarquía Persona → Cliente / Usuario, justificando explícitamente la sustitución y verificando precondiciones, postcondiciones, invariantes y excepciones en el diseño TO-BE.
D-GEN-05	LSP	Producto → Medicamento → MedicamentoCapsul a / MedicamentoLiquido	La Fase 1 no encontró una violación de LSP en la jerarquía existente. Las especializaciones conservan el comportamiento heredado.	Conservar la jerarquía farmacéutica existente. Las nuevas familias introducidas por SC-1 no se incorporarán debajo de Medicamento, sino en el nivel apropiado de la jerarquía de Producto.
D-GEN-06	DIP	ServicioProducto (Constructor)	Crea EventoStockMinimo y EventoVencimiento con new; el módulo de alto nivel queda fijado a la implementación concreta de la alarma. H-08.	Definir interfaces segregadas de notificación (INotificadorStockMinimo, INotificadorVencimiento) e inyectarlas por constructor en ServicioProducto, de manera que el servicio dependa de contratos específicos y no de clases concretas de evento. IServicioNotificacion se conserva sin conectarse al flujo porque su implementación concreta alteraría las salidas observables.

D-GEN-07	DIP	ServicioCliente (constructor)	Crea EventoPuntos con new; la notificación de acumulación de puntos queda acoplada a esa implementación concreta. H-09.	Injectar INotificadorPuntos por constructor en ServicioCliente, evitando que el servicio dependa directamente de la clase concreta EventoPuntos. La segregación de interfaces garantiza que ServicioCliente solo dependa del contrato que realmente necesita.
D-GEN-08	DIP	ServicioMovimiento (constructor)	Crea EventoMovimiento con new dentro del constructor. H-10.	Injectar INotificadorMovimiento por constructor en ServicioMovimiento, manteniendo consistencia arquitectónica con las decisiones D-GEN-06 y D-GEN-07. Los cuatro servicios quedan desacoplados de sus implementaciones concretas de notificación mediante interfaces segregadas.
D-GEN-09	DIP	ServicioProducto (carga desde archivo)	Llama a File.Exists/File.ReadAllLines directamente dentro de la lógica de negocio. H-11.	Definir una abstracción de acceso a datos e inyectarla en el servicio, de modo que la separación de persistencia ya decidida en D-SC1-01 se materialice como una dependencia hacia una interfaz, no hacia el archivo concreto.
D-GEN-10	DIP	ServicioCliente (carga desde archivo)	Llama a File.Exists/File.ReadAllLines directamente dentro de la lógica de negocio. H-12.	Aplicar el mismo mecanismo de abstracción de acceso a datos definido en D-GEN-09, complementando la separación de persistencia ya decidida en D-GEN-01.

D-GEN-11	DIP	ServicioUsuario (carga desde archivo)	Llama a File.Exists/File.ReadAllLines directamente dentro de la lógica de negocio. H-13.	Aplicar el mismo mecanismo definido en D-GEN-09 y D-GEN-10, complementando la separación de persistencia ya decidida en D-GEN-03.
D-GEN-12	ISP	IServicioNotificacion	La Fase 1 no encontró violación de ISP en IServicioNotificacion; su único método ya era pequeño y específico. Sin embargo, su única implementación concreta no puede reutilizarse sin alterar el comportamiento observable de las cuatro alertas (D-GEN-06/07/08).	Definir cuatro interfaces segregadas de notificación (INotificadorStockMinimo, INotificadorVencimiento, INotificadorPuntos, INotificadorMovimiento), una por evento, implementadas por las clases EventoX existentes. IServicioNotificacion se conserva sin conectarse al flujo: su implementación concreta imprime sin color y con un prefijo distinto, lo que alteraría cuatro salidas observables. La segregación garantiza que cada servicio dependa únicamente del contrato que le corresponde.

4. TO – BE (Decisiones seleccionadas)

4.1. Aplicación de SRP

4.1.1. Separación de responsabilidades de ServicioProducto

En el diseño original, ServicioProducto concentra la gestión de productos, la lectura desde archivos, la verificación de stock, la verificación de vencimiento y la generación de eventos. Esta concentración corresponde al hallazgo H-01.

La frontera propuesta separa estas responsabilidades de la siguiente manera:

Responsabilidad	Componente TO-BE
Gestionar la colección de productos	ServicioProducto
Obtener información desde archivos	IRepositorioProducto / RepositorioMedicamentoArchivo,

	RepositorioCosmeticoArchivo, RepositorioComestibleArchivo
Crear instancias concretas de productos	Cada repositorio crea su propio tipo
Evaluar stock	ServicioProducto
Evaluar vencimiento	ServicioProducto
Notificar eventos	INotificadorStockMinimo, INotificadorVencimiento

La decisión no consiste en crear una clase por cada método, sino en separar razones de cambio independientes. La gestión del catálogo pertenece al dominio de productos; la lectura de archivos puede cambiar por razones técnicas; la creación de nuevos tipos cambia cuando el catálogo se amplía; y el mecanismo de notificación puede cambiar independientemente de las reglas del producto.

El beneficio es que un cambio en la fuente de datos, en la forma de notificar o en la incorporación de una nueva familia de productos no obliga a modificar toda la lógica de ServicioProducto.

4.1.2. Separación de responsabilidades de ServicioCliente

ServicioCliente combina gestión de clientes, persistencia, acumulación de puntos y notificación, situación identificada como H-02.

En el TO-BE, ServicioCliente conserva la coordinación de las operaciones sobre clientes, mientras:

- La persistencia se delega a IRepositorioCliente / RepositorioClienteArchivo;
- La notificación se delega a INotificadorPuntos;
- La regla de puntos permanece como comportamiento de negocio mientras no exista una solicitud de cambio autorizada que requiera convertirla en una estrategia independiente.

Esto evita una fragmentación artificial de la clase y mantiene el límite de rediseño establecido por el equipo.

4.1.3. Separación de responsabilidades de ServicioUsuario

ServicioUsuario actualmente coordina la gestión de usuarios, la carga desde archivos y la autenticación. El equipo aceptó este hallazgo con la precisión de que la autenticación se encuentra implementada en AspectoAutenticacion y ServicioUsuario únicamente la coordina.

Por tanto, el TO-BE separa:

- Acceso a los datos de usuarios (IRepositorioUsuario / RepositorioUsuarioArchivo);
- Gestión de la colección de usuarios;
- Coordinación del proceso de autenticación.

AspectoAutenticacion se conserva para la operación de autenticación. No se introduce ninguna abstracción sobre ella; el equipo determinó en Fase 1 que es una regla fija sin variabilidad prevista.

4.1.4. Separación de presentación y dominio

Producto actualmente contiene `MostrarInformacion()`, que escribe directamente mediante `Console.WriteLine`. Este comportamiento constituye el hallazgo H-03.

El TO-BE conserva el método `MostrarInformacion()` como punto de extensión polimórfico, pero en la implementación actual `Cosmetico` y `Comestible` aún no lo sobrescriben: ambas clases heredan el comportamiento definido en `Producto` sin agregar su información específica (Marca y Calorias, respectivamente). El flujo del menú (opción 1) no invoca este método, porque el listado actual imprime Nombre, Stock y Precio en un formato propio que es comportamiento observable congelado; modificarlo cambiaría una salida que el sistema original produce hoy. Se deja registrado como pendiente de implementación: agregar el override en ambas clases no afecta ninguna salida observable, ya que el método no participa en ningún flujo ejecutado del sistema.

Esta decisión se declara como deuda técnica consciente: H-03 y H-07 quedan sin resolver completamente en esta fase.

4.2. Aplicación de OCP mediante SC-1

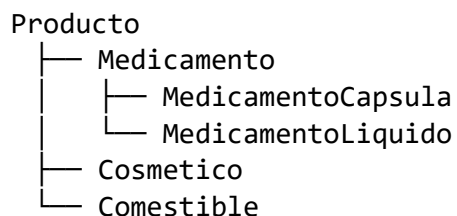
SC-1 requiere incorporar cosméticos y productos comestibles además de los productos farmacéuticos actuales. El problema principal identificado en Fase 2 está en la carga y creación de productos, ya que actualmente `CargarDesdeArchivo()` construye directamente `MedicamentoCapsula`.

El TO-BE resuelve esta situación mediante dos decisiones complementarias:

- Ampliar la jerarquía de `Producto` mediante nuevas especializaciones;
- Desacoplar la creación de productos concretos de la lógica de gestión/carga.

4.2.1. Nuevas especializaciones

La jerarquía resultante será:



`Cosmetico` y `Comestible` serán clases concretas que heredan directamente de `Producto`. No se modifica la jerarquía de `Medicamento` para introducir cosméticos o comestibles, porque estos conceptos no son medicamentos.

Esta decisión coincide con la conclusión de Fase 2: las operaciones generales de venta y stock ya trabajan sobre `Producto`, por lo que las nuevas familias pueden aprovechar esa abstracción.

4.2.2. Creación extensible mediante repositorios especializados

ProductoFactory se conserva exactamente igual que en el AS-IS. La responsabilidad de decidir qué clase concreta crear queda distribuida en tres repositorios especializados: RepositorioMedicamentoArchivo, RepositorioCosmeticoArchivo y RepositorioComestibleArchivo. Cada uno conoce el formato de su propio archivo y construye únicamente su tipo de producto.

La cadena queda: Archivo específico → Repositorio especializado → ServicioProducto.

4.2.3. Formato del archivo de productos

SC-1 se implementa mediante tres archivos separados, uno por familia de producto, en lugar de agregar una columna de tipo al archivo existente. Esta decisión garantiza retrocompatibilidad exacta: productos.txt no se modifica en ningún aspecto.

Cada archivo mantiene el formato de columnas separadas por punto y coma, propio de su familia.

Archivo	Formato y clase creada
productos.txt	nombre;precio;stock;stockMinimo;fechaVencimiento;laboratorio → MedicamentoCapsula (formato original, sin cambios)
cosmeticos.txt	nombre;precio;stock;stockMinimo;fechaVencimiento;marca → Cosmetico
comestibles.txt	nombre;precio;stock;stockMinimo;fechaVencimiento;calorias → Comestible

El archivo productos.txt original sigue funcionando sin ninguna modificación. Los cosméticos y comestibles se cargan desde sus propios archivos mediante sus repositorios especializados.

Nota de entrega: cosmeticos.txt y comestibles.txt deben crearse con el formato descrito en esta tabla antes de ejecutar el sistema; si alguno de los dos archivos no existe en el directorio de ejecución, CargarDesdeArchivo devuelve "Archivo no encontrado" para esa familia de productos sin detener el arranque del sistema, dado que RepositorioCosmeticoArchivo y RepositorioComestibleArchivo implementan Existe(ruta) antes de leer.

4.3. Aplicación de LSP

La Fase 1 no identificó una violación de LSP en las jerarquías existentes. Por tanto, el objetivo no es eliminar herencias válidas, sino conservarlas y justificar las nuevas.

4.3.1. Persona → Cliente / Usuario

Se conserva:

```
Persona
├── Cliente
└── Usuario
```

¿Por qué herencia?

Cliente y Usuario representan especializaciones de una persona y comparten los atributos definidos por Persona. La relación es estable y representa una relación “es-un”, no únicamente una necesidad de reutilización de código.

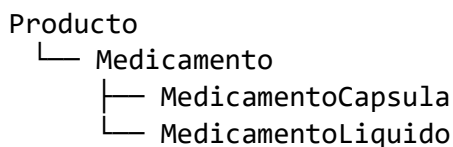
Sustituibilidad

- **Precondiciones:** Ninguna operación definida por Persona requiere condiciones adicionales en Cliente o Usuario.
- **Postcondiciones:** Una instancia de Cliente o Usuario conserva los atributos heredados definidos por Persona.
- **Invariantes:** Nombre, Cedula, Telefono y Correo continúan representando los datos comunes de una persona.
- **Excepciones:** Las subclases no introducen excepciones incompatibles con el contrato de la superclase.

Por lo tanto, la jerarquía se conserva.

4.3.2. Producto → Medicamento

Se conserva:



¿Por qué herencia?

Un medicamento es un producto y las propiedades comunes de Producto —nombre, precio, stock, stock mínimo y fecha de vencimiento— son aplicables al medicamento. Las especializaciones de medicamento agregan información propia sin invalidar el comportamiento común.

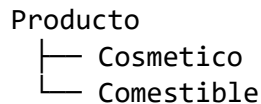
Sustituibilidad

- **Precondiciones:** las operaciones comunes se reciben bajo las mismas condiciones definidas por Producto.
- **Postcondiciones:** las operaciones heredadas mantienen sus resultados y efectos esperados.
- **Invariantes:** nombre, precio, stock, stock mínimo y fecha de vencimiento continúan siendo válidos.
- **Excepciones:** ninguna subclase exige condiciones incompatibles para utilizarse como Producto.

Por tanto, la jerarquía se conserva.

4.3.3. Nuevas especializaciones Cosmético y Comestible

Se agregan:



¿Por qué herencia y no composición?

Porque ambos conceptos son tipos de productos que deben participar directamente en las operaciones existentes que trabajan sobre Producto. Los cinco atributos de Producto tienen significado real para un cosmético y para un comestible, incluida la fecha de vencimiento. La composición obligaría a duplicar ese contrato o a introducir adaptadores donde hoy hay polimorfismo natural.

Este es el criterio opuesto al de SC-2: un servicio de inyectología no tiene stock ni vencimiento reales y se vería forzado a inventarlos. Cosmetico y Comestible no inventan nada.

Sustituibilidad

- **Precondiciones:** los constructores reciben los cinco parámetros de Producto y los delegan a base(...) sin restringirlos, añadiendo un parámetro propio.
- **Postcondiciones:** los cinco atributos heredados quedan asignados con los valores recibidos. El contrato exige que, al implementarse, MostrarInformacion() se sobrescriba invocando primero base.MostrarInformacion() y luego añadiendo una línea propia, sin reducir nunca la salida de la superclase; en el estado actual de la implementación este override todavía no se ha escrito, por lo que la postcondición queda pendiente de verificación en Fase 4.
- **Invariantes:** Stock, StockMinimo y FechaVencimiento mantienen su semántica. Las reglas $Stock \leq StockMinimo$ y $días \leq 30$ siguen siendo correctas para toda la jerarquía sin excepciones por tipo.
- **Excepciones:** ninguna subclase introduce restricciones incompatibles con el contrato de Producto.

Regla de verificación para la Fase 4: si al implementar aparece la necesidad de escribir un if (producto is Cosmetico) o similar fuera de los repositorios especializados, la herencia está mal planteada y hay que revisar este diseño

4.4. Aplicación de DIP

Las decisiones DIP se materializan mediante abstracciones inyectadas por constructor.

4.4.1. Persistencia

Los módulos de alto nivel son ServicioProducto, ServicioCliente y ServicioUsuario. Los módulos de bajo nivel son las implementaciones que leen archivos.

Abstracción	Implementación	Cliente
IRepositorioProducto	Cada repositorio crea su propio tipo	ServicioProducto
IRepositorioCliente	RepositorioClienteArchivo	ServicioCliente
IRepositorioUsuario	RepositorioUsuarioArchivo	ServicioUsuario

Cada interfaz expone dos operaciones: Existe(ruta): bool y Leer...(ruta): IEnumerable<T>. Los servicios dejan de conocer File.Exists() y File.ReadAllLines().

Sobrecarga de carga para repositorios adicionales

ServicioProducto conserva un único constructor con IRepositorioProducto inyectado (el repositorio de medicamentos, para no alterar la carga de productos.txt), y expone además una sobrecarga CargarDesdeArchivo(IRepositorioProducto repo, string ruta) que recibe el repositorio como parámetro. Esta sobrecarga es el mecanismo mediante el cual Program inyecta RepositorioCosmeticoArchivo y RepositorioComestibleArchivo sobre la misma instancia de ServicioProducto, sin duplicar el servicio ni introducir un ServicioProducto por familia de producto. La dependencia sigue siendo hacia IRepositorioProducto en ambos casos; cambia únicamente el punto de inyección, de constructor a parámetro de método, para permitir que un mismo servicio cargue varias fuentes de datos compatibles con su misma abstracción.

4.4.2. Notificaciones

Los módulos de alto nivel son ServicioProducto, ServicioCliente y ServicioMovimiento. Los módulos de bajo nivel son las clases EventoX existentes.

La abstracción no es IServicioNotificacion. Su única implementación concreta (ServicioNotificacion) imprime [NOTIFICACION] {mensaje} sin color, mientras que las cuatro alertas actuales tienen texto y color propios definidos en Program.cs. Conectar ServicioNotificacion al flujo cambiaría cuatro salidas observables.

La inversión se realiza mediante cuatro interfaces mínimas, una por evento. Cada EventoX ya existente implementa su interfaz:

Abstracción	Implementación	Cliente
INotificadorStockMinimo	EventoStockMinimo	ServicioProducto
INotificadorVencimiento	EventoVencimiento	ServicioProducto
INotificadorPuntos	EventoPuntos	ServicioCliente
INotificadorMovimiento	EventoMovimiento	ServicioMovimiento

ServicioProducto —→ INotificadorStockMinimo ◁— EventoStockMinimo
 ServicioProducto —→ INotificadorVencimiento ◁— EventoVencimiento
 ServicioCliente —→ INotificadorPuntos ◁— EventoPuntos
 ServicioMovimiento —→ INotificadorMovimiento ◁— EventoMovimiento

Los servicios dejan de ejecutar new EventoX() en su constructor y dejan de exponer el evento como campo público. IServicioNotificacion y ServicioNotificacion se conservan en el diseño sin conectarse al flujo, igual que en el AS-IS.

4.5. Aplicación de ISP

ISP se aplica en dos lugares con alternativa descartada explícita.

Repositorios. Alternativa descartada: una única interfaz IRepositorio con todas las operaciones de lectura. Con ella, ServicioUsuario —el módulo que maneja credenciales— dependería de LeerProductos() y LeerClientes(), que jamás usa. Un cambio en la firma de LeerProductos obligaría a revisar el servicio de usuarios sin ninguna razón. Por eso se definen tres interfaces, una por agregado, con exactamente las operaciones que su cliente utiliza.

Notificadores. Alternativa descartada: una única interfaz INotificador con las cuatro operaciones de notificación. Con ella, ServicioMovimiento dependería de NotificarStockMinimo() y NotificarVencimiento(), que no le corresponden. Por eso se definen cuatro interfaces de una sola operación.

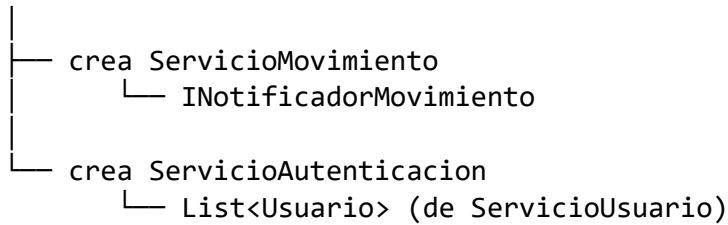
IServicioNotificacion se conserva tal como está (un único método). No se integra al flujo, pero su contrato ya es correcto desde el punto de vista de ISP.

4.6. Composition Root

Program es el punto donde se ensamblan todas las dependencias concretas. Es el único lugar del sistema donde se escribe new sobre clases de infraestructura.

Program

```
|
|— crea EventoStockMinimo → suscribe delegado rojo
|— crea EventoVencimiento → suscribe delegado amarillo
|— crea EventoPuntos      → suscribe delegado verde
|— crea EventoMovimiento  → suscribe delegado cian
|
|— crea RepositorioMedicamentoArchivo
|— crea RepositorioCosmeticoArchivo
|— crea RepositorioComestibleArchivo
|— crea RepositorioClienteArchivo
|— crea RepositorioUsuarioArchivo
|
|— crea ServicioProducto
|   |— IRepositorioProducto
|   |— INotificadorStockMinimo
|   |— INotificadorVencimiento
|
|— crea ServicioCliente
|   |— IRepositorioCliente
|   |— INotificadorPuntos
|
|— crea ServicioUsuario
|   |— IRepositorioUsuario
```



Los delegados se suscriben antes de inyectar los notificadores. El orden de las salidas por consola no cambia: la primera salida sigue siendo “Cargando información del sistema...”.

4.7. Inventario definitivo para el UML TO-BE

4.7.1. Entidades y jerarquías conservadas (negro)

- Persona, Cliente, Usuario
- Producto, Medicamento, MedicamentoCapsula, MedicamentoLiquido
- Laboratorio, Movimiento
- TipoRelleno (enum), MaterialEnvase (enum)

4.7.2. Nuevas clases de SC-1

- Cosmetico (atributo propio: Marca : string)
- Comestible (atributo propio: Calorias : int)

4.7.3. Servicios modificados

- ServicioProducto — pierde acceso a archivo y creación de eventos con new
- ServicioCliente — pierde acceso a archivo y creación de evento con new
- ServicioUsuario — pierde acceso a archivo y delega la autenticación a ServicioAutenticacion
- ServicioMovimiento — pierde creación de evento con new

4.7.4. Interfaces nuevas

- IRepositoryProducto, IRepositoryCliente, IRepositoryUsuario
- INotificadorStockMinimo, INotificadorVencimiento, INotificadorPuntos, INotificadorMovimiento

4.7.5. Clases nuevas de infraestructura

- RepositorioMedicamentoArchivo, RepositorioCosmeticoArchivo, RepositorioComestibleArchivo, RepositorioClienteArchivo, RepositorioUsuarioArchivo

4.7.6. Clases de eventos modificadas

EventoStockMinimo, EventoVencimiento, EventoPuntos, EventoMovimiento: cada una implementa su interfaz de notificador correspondiente. El texto de los mensajes y los delegados no cambian.

4.7.7. Conservadas sin intervención (negro, sin uso activo)

- IServicioNotificacion, ServicioNotificacion
- IDescuento, ServicioDescuento
- AspectoAutenticacion, AspectoValidacion

4.7.8. Composition root

- Program

4.7.9. Nuevo servicio

- ServicioAutenticacion — clase nueva que recibe la lista de usuarios de ServicioUsuario y coordina el login delegando en AspectoAutenticacion.

4.8. Relaciones que debe representar el UML

Herencia

```

Persona    <|-- Cliente
Persona    <|-- Usuario
Producto   <|-- Medicamento
Medicamento <|-- MedicamentoCapsula
Medicamento <|-- MedicamentoLiquido
Producto   <|-- Cosmetico
Producto   <|-- Comestible

```

Implementación de interfaz

```

RepositorioMedicamentoArchivo ..|> IRepositorioProducto
RepositorioCosmeticoArchivo    ..|> IRepositorioProducto
RepositorioComestibleArchivo    ..|> IRepositorioProducto
RepositorioClienteArchivo       ..|> IRepositorioCliente
RepositorioUsuarioArchivo       ..|> IRepositorioUsuario
EventoStockMinimo               ..|> INotificadorStockMinimo
EventoVencimiento               ..|> INotificadorVencimiento

```

EventoPuntos	.. > INotificadorPuntos
EventoMovimiento	.. > INotificadorMovimiento
ServicioNotificacion	.. > IServicioNotificacion (conservada, sin uso en flujo)
ServicioDescuento	.. > IDescuento (conservada, sin uso en flujo)

Dependencias por inyección de constructor

ServicioProducto --> IRepositoryProducto
 ServicioProducto --> INotificadorStockMinimo
 ServicioProducto --> INotificadorVencimiento
 ServicioCliente --> IRepositoryCliente
 ServicioCliente --> INotificadorPuntos
 ServicioUsuario --> IRepositoryUsuario
 ServicioMovimiento --> INotificadorMovimiento
 ServicioAutenticacion --> List<Usuario> (recibida de ServicioUsuario.ObtenerUsuarios())

Asociaciones existentes (negro)

Medicamento --> Laboratorio
 Movimiento --> Producto

Relaciones que NO deben aparecer

- ServicioProducto → ProductoFactory (ProductoFactory no participa en el flujo de creación de productos)
- ServicioX → EventoX concreto
- ServicioX → IServicioNotificacion o ServicioNotificacion
- Cosmetico → Laboratorio o Comestible → Laboratorio
- Cualquier clase de SC-2 o SC-3

5. Estrategia de pruebas y comportamiento

5.1. Objetivo de las pruebas

El objetivo de las pruebas es comprobar que el rediseño de la arquitectura mantiene el comportamiento observable del sistema original. Aunque internamente se realizan cambios en la organización de responsabilidades, dependencias y creación de objetos, las funcionalidades que ya existían no deberían presentar cambios no autorizados.

Entonces, durante la Fase 1 se realizaron las pruebas correspondientes al sistema original y se documentaron sus resultados mediante las evidencias respectivas entregadas en el documento 1, que se evidencia en el repositorio carpeta fase1. Por lo que estos resultados se tomarán como referencia o línea base para la validación del sistema rediseñado.

Por esta razón, en esta fase no se repetirán las pruebas del sistema original. En la Fase 4, una vez implementado el nuevo diseño, se utilizarán las mismas entradas y condiciones para comprobar que el comportamiento del sistema TO-BE se mantiene respecto a la línea base establecida.

La estrategia se enfoca principalmente en las funcionalidades identificadas durante las fases anteriores y que podrían verse afectadas por las modificaciones realizadas en el TO-BE: carga de productos, listado, búsqueda, venta, actualización de stock, alertas de stock y vencimiento, movimientos, clientes, puntos y autenticación.

Además, se tendrán en cuenta casos límite para comprobar posteriormente que los cambios arquitectónicos no alteren las reglas que ya tenía el sistema.

5.2. Criterios de comparación

Para determinar posteriormente si el comportamiento fue preservado, se compararán los resultados del sistema TO-BE con los resultados obtenidos y documentados para el sistema AS-IS en la Fase 1.

Los principales aspectos que se tendrán en cuenta son:

- Datos mostrados al usuario.
- Valores calculados.
- Cantidades de stock antes y después de una operación.
- Mensajes generados por el sistema.
- Alertas de stock mínimo y vencimiento.
- Registros de movimientos.
- Resultados de búsquedas.
- Acumulación de puntos.
- Resultado de la autenticación.
- Manejo de entradas válidas y entradas límite.
- Orden de las salidas cuando este sea observable.

5.3. Casos de prueba definidos

A partir de las pruebas realizadas durante la Fase 1, se seleccionaron 10 casos de prueba para revisar tanto el funcionamiento habitual del sistema como situaciones en las que se llega a una condición especial.

- **Normal:** corresponde a una situación de uso habitual, en la que se utilizan datos válidos y se espera que el sistema funcione de la manera acostumbrada.
Ejemplo: buscar un producto que sí existe.
- **Límite:** corresponde a una situación cercana a una condición especial del sistema, donde se quiere comprobar que las reglas existentes se mantienen correctamente.
Ejemplo: vender exactamente la cantidad de unidades que hay disponibles en stock.

También puede aparecer **Normal / límite** cuando el mismo caso permite comprobar ambas situaciones, como ocurre con el inicio de sesión con credenciales válidas e inválidas.

ID	Funcionalidad	Tipo	Entrada / condición	Resultado esperado
CP-01	Carga de productos	Normal	Ejecutar el sistema con el archivo productos.txt original	Se deben cargar los mismos productos y conservar la información observable registrada en el AS-IS.
CP-02	Listado de productos	Normal	Seleccionar la opción de listar productos	Se deben mostrar los mismos productos y datos observables del sistema original.
CP-03	Búsqueda de producto	Normal	Buscar un producto existente por nombre	Se debe obtener el mismo producto y la misma información que en el sistema original.
CP-04	Búsqueda sin coincidencias	Límite	Buscar un nombre que no exista	Se debe conservar el mismo comportamiento y mensaje del sistema original.
CP-05	Registro de venta	Normal	Seleccionar un producto existente y vender una cantidad válida	Se debe descontar la misma cantidad de stock y registrar el movimiento de forma equivalente.
CP-06	Venta en límite de stock	Límite	Vender una cantidad igual al stock disponible	Se debe conservar el comportamiento del sistema original respecto al stock y al movimiento generado.
CP-07	Alerta de stock mínimo	Límite	Utilizar un producto cuyo stock sea igual o menor al stock mínimo	Se debe generar la misma alerta observable registrada en el AS-IS.
CP-08	Alerta de vencimiento	Límite	Utilizar un producto cuya fecha de vencimiento se encuentre dentro del rango de alerta	Se debe conservar la misma alerta y comportamiento observable.
CP-09	Acumulación de puntos	Normal	Registrar una cantidad de puntos para un cliente existente	El total de puntos debe coincidir con el resultado registrado en el AS-IS.
CP-10	Autenticación	Normal / límite	Intentar iniciar sesión con credenciales válidas y posteriormente con credenciales inválidas	El resultado de ambos intentos debe coincidir con el comportamiento registrado en el AS-IS.

Registros De Decisión Arquitectónica

Una decisión sin alternativa descartada no es una decisión: es la primera idea que surgió. Por ello, cada ADR incluye al menos dos alternativas razonables, una de las cuales fue rechazada con argumento explícito. El objetivo es que un revisor estricto pueda reconstruir el razonamiento sin necesidad de inferirlo.

ADR-01 · Separación de responsabilidades en ServicioProducto

Estado: Aceptada

Contexto y evidencia

En el AS-IS, la clase ServicioProducto concentra la gestión del catálogo de productos, la lectura directa desde archivos de texto (File.Exists / File.ReadAllLines), la verificación de stock mínimo, la verificación de vencimiento y la creación e invocación de eventos de notificación. Este acoplamiento de razones de cambio independientes corresponde al hallazgo H-01 (severidad alta) del inventario de la Fase 1.

Cualquier cambio en la fuente de datos, en la política de alertas o en el mecanismo de notificación obliga a modificar y reprobador la misma clase, elevando el riesgo de regresión sobre comportamientos que hoy funcionan correctamente (venta, listado, búsqueda).

Alternativas evaluadas

Alternativa A: Mantener ServicioProducto monolítico y solo agregar comentarios o regiones internas. Costo de implementación nulo, pero no reduce el riesgo de regresión ni prepara el sistema para SC-1 (nuevos tipos de producto).

Alternativa B: Extraer cada método a una clase distinta (ServicioCarga, ServicioStock, ServicioVencimiento, ServicioNotificacionProducto). Fragmentación excesiva: se multiplica el número de tipos sin una frontera de cambio clara y se complica el ensamblaje sin beneficio proporcional.

Alternativa C: Separar por razón de cambio: (1) gestión del catálogo permanece en ServicioProducto; (2) persistencia se delega a IRepositorioProducto + implementaciones de archivo; (3) notificación se delega a interfaces segregadas INotificadorStockMinimo e INotificadorVencimiento inyectadas por constructor.

Decisión

Se adopta la Alternativa C. ServicioProducto conserva la coordinación del dominio (colección, verificación de reglas de inventario). La lectura de archivos y la creación de instancias concretas de producto se delegan a repositorios. La emisión de alertas se realiza a través de abstracciones de notificación inyectadas.

Costo / consecuencia negativa aceptada

Se introduce un mayor número de tipos (interfaces e implementaciones de repositorio y notificador) y se obliga a Program a actuar como composition root. El equipo acepta este costo de ensamblaje a

cambio de que un cambio en la fuente de datos o en la forma de notificar no obligue a modificar ni a reprobear la lógica de negocio de ServicioProducto.

Principios involucrados

SRP (razones de cambio independientes) · DIP (el módulo de alto nivel depende de abstracciones, no de File.* ni de EventoX concretos).

ADR-02 · Creación extensible de productos mediante repositorios especializados

Estado: Aceptada

Contexto y evidencia

El método ServicioProducto.CargarDesdeArchivo() construye siempre un MedicamentoCapsula a partir de cada línea de productos.txt, sin condición de tipo. El hallazgo H-05 (OCP) y el análisis de impacto de SC-1 identifican este punto como el principal riesgo: incorporar cosméticos o comestibles exigiría modificar una lógica que ya funciona.

ProductoFactory existe en el código, pero es estática, no se invoca desde ningún flujo real y no resuelve la lectura desde archivo.

Alternativas evaluadas

Alternativa A: Agregar una columna «tipo» a productos.txt y un switch/if dentro de CargarDesdeArchivo() que instancie MedicamentoCapsula, Cosmetico o Comestible según el valor. Cumple el requisito funcional, pero viola OCP: cada nueva familia obliga a editar el método central de carga.

Alternativa B: Reactivar ProductoFactory como clase de instancia, inyectarla y centralizar en ella CrearDesdeDatos(string[]). Una sola fábrica conoce todos los tipos. Extensible, pero concentra el conocimiento de formatos de archivo distintos en un único punto; un error de formato de cosméticos podría afectar la carga de medicamentos.

Alternativa C: Un repositorio especializado por familia (RepositorioMedicamentoArchivo, RepositorioCosmeticoArchivo, RepositorioComestibleArchivo), todos implementando IRepositoryProducto. Cada uno conoce únicamente el formato de su archivo y construye solo su tipo. ServicioProducto recibe el repositorio de medicamentos por constructor y, mediante una sobrecarga CargarDesdeArchivo(IRepositorioProducto, ruta), carga las demás familias sin duplicar el servicio.

Decisión

Se adopta la Alternativa C. productos.txt permanece intacto (retrocompatibilidad exacta). cosmeticos.txt y comestibles.txt se leen por repositorios propios. ProductoFactory se conserva en el diseño como código huérfano (igual que en el AS-IS), sin participar en el flujo de creación.

Costo / consecuencia negativa aceptada

Existen tres clases de repositorio de producto en lugar de una. El equipo acepta esta multiplicación porque cada repositorio tiene una única razón de cambio (el formato de su familia) y porque agregar una cuarta familia no obliga a modificar los repositorios existentes ni a ServicioProducto.

Principios involucrados

OCP (extensión por nuevos repositorios, sin modificar los existentes) · DIP (ServicioProducto depende de IRepositorioProducto) · ISP (la interfaz de repositorio de producto no se mezcla con la de clientes o usuarios).

ADR-03 · Cosmético y Comestible como especializaciones directas de Producto

Estado: Aceptada

Contexto y evidencia

SC-1 introduce cosméticos y productos comestibles. Estos conceptos no son medicamentos: no tienen laboratorio farmacéutico ni tipo de relleno/material de envase. Forzarlos bajo Medicamento contaminaría la jerarquía farmacéutica y violaría el significado del dominio (hallazgo de impacto SC-1 y decisión D-SC1-03).

La Fase 1 no registró violación de LSP en Producto → Medicamento → MedicamentoCapsula/MedicamentoLiquido. El objetivo es conservar esa jerarquía y justificar las nuevas ramas.

Alternativas evaluadas

Alternativa A: Modelar Cosmetico y Comestible por composición (un Producto «envuelto»). Obligaría a adaptar todas las operaciones que hoy reciben Producto (venta, stock, movimientos) o a duplicar el contrato de los cinco atributos comunes.

Alternativa B: Cosmetico y Comestible heredan directamente de Producto. Los cinco atributos de Producto (nombre, precio, stock, stock mínimo, fecha de vencimiento) tienen significado real para ambas familias, incluida la fecha de vencimiento. La herencia expresa una relación «es-un» genuina.

Alternativa C: Hacer que Cosmetico y Comestible hereden de Medicamento, dejando Laboratorio en null o con valores dummy. Compila, pero introduce atributos sin semántica real y abre la puerta a NullReferenceException o a reglas de negocio incorrectas (LSP debilitado).

Decisión

Se adopta la Alternativa B. La jerarquía resultante es Producto ← Medicamento ← (MedicamentoCapsula | MedicamentoLiquido) y, en paralelo, Producto ← (Cosmetico | Comestible). Ninguna operación existente que trabaje sobre Producto requiere modificación para aceptar las nuevas familias.

Costo / consecuencia negativa aceptada

MostrarInformacion() aún no se sobrescribe en Cosmetico ni en Comestible (deuda técnica consciente vinculada a H-03/H-07). El listado del menú no invoca ese método; por tanto, el comportamiento observable no cambia. El equipo acepta dejar el override pendiente para la Fase 4, con la regla de verificación: si aparece un if (producto is Cosmetico) fuera de los repositorios, la herencia está mal planteada.

Principios involucrados

OCP (nuevas familias por extensión) · LSP (sustituibilidad: constructores delegan a base(...) sin restringir precondiciones; invariantes de stock y vencimiento se preservan).

ADR-04 · Interfaces segregadas de notificación (no reutilizar IServicioNotificacion)

Estado: Aceptada

Contexto y evidencia

Los constructores de ServicioProducto, ServicioCliente y ServicioMovimiento crean con new las clases EventoStockMinimo, EventoVencimiento, EventoPuntos y EventoMovimiento (hallazgos H-08, H-09, H-10). El módulo de alto nivel queda fijado a la implementación concreta de la alarma.

Existe IServicioNotificacion con un único método EnviarNotificacion(string). Su implementación concreta (ServicioNotificacion) imprime «[NOTIFICACION] {mensaje}» sin color. Las cuatro alertas del AS-IS tienen texto y color propios definidos en Program.cs. Conectar ServicioNotificacion al flujo alteraría cuatro salidas observables.

Alternativas evaluadas

Alternativa A: Cuatro interfaces mínimas de una sola operación (INotificadorStockMinimo, INotificadorVencimiento, INotificadorPuntos, INotificadorMovimiento). Cada EventoX existente implementa su interfaz correspondiente. Los servicios reciben por constructor solo el contrato que necesitan.

Alternativa B: Inyectar IServicioNotificacion en los tres servicios y adaptar los mensajes. Viola el requisito de preservar comportamiento observable: se pierden los colores y el formato exacto de las alertas.

Alternativa C: Una única interfaz INotificador con los cuatro métodos (NotificarStockMinimo, NotificarVencimiento, NotificarPuntos, NotificarMovimiento). Viola ISP: ServicioMovimiento dependería de operaciones de stock y vencimiento que no le corresponden.

Decisión

Se adopta la Alternativa A. IServicioNotificacion y ServicioNotificacion se conservan en el diseño sin conectarse al flujo (código huérfano, igual que en el AS-IS). Los delegados de color se suscriben en Program sobre las instancias concretas de EventoX antes de inyectarlas como sus interfaces.

Costo / consecuencia negativa aceptada

Cuatro interfaces en lugar de una. El equipo acepta este costo porque cada servicio depende únicamente de lo que usa, y porque un cambio en la firma de `NotificarStockMinimo` no obliga a revisar `ServicioMovimiento` ni `ServicioCliente`.

Principios involucrados

DIP (inversión hacia abstracciones) · ISP (interfaces segregadas por cliente) · preservación de comportamiento observable.

ADR-05 · Abstracción de persistencia por agregado (IRepositorio*)

Estado: Aceptada

Contexto y evidencia

`ServicioProducto`, `ServicioCliente` y `ServicioUsuario` invocan `File.Exists` y `File.ReadAllLines` directamente dentro de la lógica de negocio (hallazgos H-11, H-12, H-13). La lógica de dominio queda atada a un formato de archivo concreto; cambiar la fuente de datos obliga a modificar los servicios.

Alternativas evaluadas

Alternativa A: Una única interfaz `IRepositorio` con `LeerProductos`, `LeerClientes` y `LeerUsuarios`. Viola ISP: `ServicioUsuario` (módulo de credenciales) dependería de operaciones de productos y clientes que jamás usa. Un cambio en `LeerProductos` obligaría a revisar el servicio de autenticación sin razón.

Alternativa B: Acceso a archivos encapsulado en métodos estáticos de utilidad (`ArchivoHelper`). Reduce duplicación de `File.*`, pero no invierte la dependencia: los servicios siguen dependiendo de un detalle técnico concreto y no se puede sustituir la fuente en pruebas ni en evolución futura.

Alternativa C: Tres interfaces segregadas por agregado (`IRepositorioProducto`, `IRepositorioCliente`, `IRepositorioUsuario`), cada una con `Existe(ruta)` y `Leer...(ruta)` que retorna `IEnumerable<T>` con evaluación perezosa (`yield return`). Implementaciones de archivo inyectadas por constructor.

Decisión

Se adopta la Alternativa C. Los servicios dejan de conocer `System.IO`. La evaluación perezosa preserva el comportamiento observable ante fallos parciales de lectura (los registros ya leídos permanecen en la colección, igual que en el AS-IS).

Costo / consecuencia negativa aceptada

Tres interfaces e igual número de implementaciones de archivo. El equipo acepta la multiplicación porque cada interfaz tiene exactamente las operaciones que su cliente utiliza, y porque la sustitución

de la fuente de datos (por ejemplo, a base de datos) se localiza en las implementaciones sin tocar los servicios.

Principios involucrados

DIP · ISP · SRP (la razón de cambio de la persistencia queda fuera del servicio de dominio).

ADR-06 · Extracción de la coordinación de autenticación

Estado: Aceptada

Contexto y evidencia

ServicioUsuario administra la colección de usuarios, carga datos desde archivo y coordina el login (hallazgo H-04). El equipo aceptó el hallazgo con la precisión de que la autenticación ya está implementada en AspectoAutenticacion; ServicioUsuario solo la invoca. Aun así, la clase mezcla custodia de la colección, persistencia y coordinación del proceso de autenticación.

Alternativas evaluadas

Alternativa A: Extraer ServicioAutenticacion como clase de coordinación que recibe la lista de usuarios (obtenida de ServicioUsuario.ObtenerUsuarios()) y delega en AspectoAutenticacion. ServicioUsuario queda con una única responsabilidad: custodiar la colección y su carga. AspectoAutenticacion se conserva sin cambios.

Alternativa B: Dejar Login() dentro de ServicioUsuario e inyectar solo el repositorio. Reduce el alcance del cambio, pero mantiene dos razones de cambio en la misma clase (gestión de colección vs. coordinación de autenticación).

Alternativa C: Introducir una interfaz IAutenticacion e inyectarla. El equipo determinó en Fase 1 que AspectoAutenticacion es una regla fija sin variabilidad prevista; forzar una abstracción no aporta valor y añade complejidad innecesaria (decisión ya documentada al refutar la sugerencia de DIP sobre el aspecto estático).

Decisión

Se adopta la Alternativa A. La autenticación se coordina desde ServicioAutenticacion. ServicioUsuario no expone Login; el composition root obtiene la lista de usuarios y la entrega al servicio de autenticación. AspectoAutenticacion permanece como regla estática de dominio.

Costo / consecuencia negativa aceptada

Una clase adicional y un paso más en el composition root. El equipo acepta este costo porque separa de forma explícita la custodia de credenciales de la orquestación del login, y porque no introduce abstracciones artificiales sobre una regla que el propio equipo declaró sin variabilidad.

Principios involucrados

SRP (una razón de cambio por clase) · respeto a la decisión de Fase 1 de no abstraer AspectoAutenticacion.

Notas del diagrama UML TO-BE

Composition Root: único punto del sistema donde se escribe "new" sobre clases concretas de infraestructura.

Crea: EventoStockMinimo, EventoVencimiento, EventoPuntos, EventoMovimiento (y suscribe sus delegados de color ANTES de inyectarlos como notificadores); RepositorioMedicamentoArchivo, RepositorioCosmeticoArchivo, RepositorioComestibleArchivo, RepositorioClienteArchivo, RepositorioUsuarioArchivo; ServicioProducto, ServicioCliente, ServicioUsuario, ServicioMovimiento; y ServicioAutenticacion (recibe la lista ya cargada por ServicioUsuario.ObtenerUsuarios()).

ServicioProducto.CargarDesdeArchivo() se invoca 3 veces: una con su repositorio inyectado (productos.txt) y dos más con el overload que recibe un IRepositorioProducto distinto (cosmeticos.txt, comestibles.txt) — así se agregan familias de producto sin modificar el servicio.